Does Deciding at Write Time Help? KV Cache Placement and Precision Under Read-Adaptive Restore

Anonymous Author(s)
Submission under review

***Abstract*—A user who returns to a long LLM conversation waits while the serving system restores the session's KV cache from CPU memory, from an SSD, or by recomputation. Recent systems settle two properties of each KV block when they write it, the tier that holds it and the precision it keeps, and they often key both on token position. Restore has meanwhile become adaptive: Cake recomputes the prefix on the GPU while it loads the suffix from storage, and the two streams meet at a point that only exists at read time. We ask whether a write-time decision still buys anything that a later decision cannot, and we answer with two tests on an AMD MI300X. A deferral test compares each write-time placement rule with the same rule applied at eviction, in the background, or by pre-cleaning: in 36 GPU comparisons, no write-time rule beat all of its deferred twins by 5%, because position stays visible at eviction. Read accounting fixes bytes, codec, and attention path, moves only the low-precision blocks, and logs which blocks each restore actually loads. On slow tiers Cake almost never reads low-precision front blocks, so their harmlessness says nothing about sensitivity. Under a forced full load, uniform FP8 keeps Llama-3.1-8B within 0.5 points of BF16; Qwen2.5-7B loses 32.8 points and recovers fully once its keys are mean-centered, while sink protection recovers at most 0.5. Uniform INT4 meets a 2-point bound on both models, and 25% INT4 placed in front, middle, or back stays within 0.33 points of random placement. Both decisions can wait until demotion or read time.**

I. Introduction

Long LLM conversations pause and resume. When a user returns, the serving system must rebuild the KV cache of the whole earlier context before it emits the first token. For Llama-3.1-8B [1], a 32K-token context holds 4 GiB of KV in BF16, more than GPU memory can keep for every idle session. Serving systems therefore move idle KV to CPU memory or SSD and restore it when the user returns [2], [3], [4], [5]. The restore time adds directly to the returning request's time to first token (TTFT).

Every stored KV block carries two decisions: the tier that holds it and the precision it keeps. Several recent designs make both decisions when they write the block, and many key on token position. Krul compresses a conversation's KV when the conversation goes idle, keeps a hardware-calibrated share of each layer, and recomputes the rest on restore [6]. TierKV splits each request's tokens by position into uncompressed, low-rank, and flash-resident ranges before decoding starts [7]. Lachesis places context segments in HBM or flash when it allocates them [8]. On the precision side, KV quantizers keep the first few "sink" tokens and a window of recent tokens at high precision [9], [10], [11], [12].

Restore has become adaptive at the other end. Cake recomputes blocks from the start of the context on the GPU while it loads blocks from the end over I/O, and the two streams meet where they cross [13]. The meeting point moves with GPU load and tier bandwidth, so it exists only at read time. CacheGen similarly chooses a compression level for each chunk when it sends that chunk, from the bandwidth it measures at that moment [14].

This combination raises a question that prior work leaves open: *when the reader adapts, does deciding at write time buy anything that a later decision cannot?* We answer it in two steps, one per decision.

Step 1: placement. Write-time placement designs compare against position-blind baselines and rarely against the same rule applied later, at eviction or in the background. A write-time rule that beats LRU may therefore owe its whole gain to the rule, with timing playing no part. The *deferral test* (§ III-C) closes this gap: a write-time rule must beat each of its *deferred twins* by at least 5% on the GPU, namely the same rule applied when the CPU tier fills, the same rule applied in the background, a pre-cleaning copy, and write-through. No write-time rule passes, because token position never changes and the deferred twin sees it at eviction as well.

Step 2: precision. Once placement can wait, precision becomes the strongest remaining case for deciding early: a block stored in FP8 cannot return to BF16 without a second copy, and quantizers commonly protect tokens by position. Existing evaluations of position-aware precision, however, ignore which blocks the reader loads. Under Cake the GPU recomputes the prefix in exact BF16 whatever copy sits in storage, so a low-precision front that costs no accuracy may be insensitive, or the restore may never read it. *Read accounting* (§ III-D) separates the two explanations. It fixes the stored bytes, the codec, and the attention path, changes only where the low-precision blocks sit, and logs for every block whether the restore recomputed it, loaded it in BF16, or loaded its low-precision copy. Two read modes then separate sensitivity from read probability: a forced full load exposes every stored copy, and a real Cake restore reads only the copies past the meeting point.

We run both steps on one AMD MI300X with a KV harness that checks every restore bit for bit, and we commit decision

rules and analysis code before we collect any result they judge.

This paper makes four contributions.

1) **Placement can wait (§ IV).** In 36 GPU comparisons, no write-time placement rule beat all of its deferred twins by 5%. The strongest write-time rule ran 18.9% faster than the best position-blind policy but only 0.7% faster than its own deferred twin, because position stays visible at eviction. Deferral loses only under a sustained SSD write backlog with a bounded queue, which a single GPU does not produce: a pre-registered rerun on ten fresh seeds found 0 of 216 winning cells.
2) **Read accounting separates "insensitive" from "unread" (§ V-B).** On rate-limited tiers, Cake reads 0 of 16 low-precision front blocks at the balanced meeting point and all 16 back blocks. On the real CPU tier it meets after 1–3 blocks and reads almost every copy.
3) **Precision does not need position (§ V).** Under a forced full load, uniform FP8 keeps Llama-3.1-8B within 0.5 points of BF16. Qwen2.5-7B-1M loses 32.8 points; protecting sink and recent blocks recovers at most 0.5 points, while mean-centering its keys before FP8, or storing INT8, recovers all of it. Uniform INT4 meets a 2-point bound on both models, and 25% INT4 placed in the front, middle, or back stays within 0.33 points of random placement.
4) **Low precision buys time only on slow tiers (§ V-A).** INT4 cuts restore time by 36–57% on rate-limited tiers but slows the real CPU tier by up to 2.2×, because dequantization competes with recomputation on the GPU.

Together the two steps give one answer. When restore adapts at read time, a system can write every block to CPU memory in BF16, defer demotion and quantization until space runs out, and let the reader choose what to recompute. Choosing a low-precision format is a question of codec and ratio, and position adds nothing we can measure. Our evidence covers one GPU type, two GQA models, 32K-token contexts for quality, rate-limited slow tiers, and a single-request GPU harness; § VIII lists what these limits leave open.

II. Background

Two facts set up the question: restore time sits on the returning request's critical path, and Cake fixes the split between recomputation and loading only at read time.

A. Restoring a returning session

We split a context into *blocks* of 512 tokens. A Llama-3.1-8B block holds 64 MiB of BF16 KV (128 KiB per token) and a Qwen2.5-7B block holds 28 MiB, because Qwen has 28 layers with 4 KV heads where Llama has 32 layers with 8. When a session goes idle, the system keeps its blocks in a fast but small tier (CPU DRAM), a slow but large tier (local SSD or network storage), or nowhere, in which case it recomputes them on return. We call the request that resumes a session the *returning request* and report its TTFT.

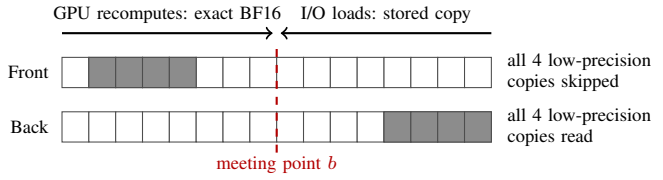


Fig. 1. **Recomputation hides stored precision.** Gray blocks hold low-precision copies; block 0 holds the attention sink. Blocks before the meeting point b come back as exact BF16, so a low-precision front costs no accuracy because no one reads it. b moves with GPU load and tier bandwidth.

B. Read-adaptive restore

Cake restores n blocks with two concurrent streams [13]. The GPU recomputes blocks $0, 1, 2, \ldots$ and the I/O path loads blocks $n{-}1, n{-}2, \ldots$; the streams stop where they meet, at block b. Blocks before b come from recomputation and blocks from b on come from storage. Recomputing block i costs $f(i)$, which grows with i because attention covers the whole prefix. On our platform f rises from 28.9 ms for the first block to 98.8 ms at block 80, about 0.89 ms per block. Loading a block costs ℓ, set by tier bandwidth: 1.76 ms from CPU memory at 35.4 GiB/s and 188.8 ms from a 0.331 GiB/s network tier. Given f and the tier bandwidth, a simple two-pointer simulation predicts the measured meeting point within one block in 28 of 28 cells we tested. The meeting point therefore depends on two quantities that only the reader sees: how busy the GPU is and how fast the tier delivers.

C. Two write-time decisions and their deferred twins

A write-time *placement* decision assigns a block to a tier when the system writes the block. Its deferred twin writes the block to the CPU tier and applies the same assignment when the CPU tier fills. The twin sees the same token position, because position never changes after the write. A write-time *precision* decision stores the block in a low-precision format when it writes the block. Its deferred twin keeps BF16 in the CPU tier and quantizes the block when it demotes the block to a slower tier. Both orders reach the same final layout; the deferred order costs some extra CPU space until demotion and gives an early return a BF16 copy.

D. Why recomputation hides precision

Cake's split creates a confound for any position-aware precision rule (Fig. 1). Recomputation regenerates a block from tokens, so a recomputed block is exact BF16 regardless of the stored copy. Only blocks past the meeting point expose their stored precision to the model. A configuration that stores the front in low precision and shows no accuracy loss under Cake therefore admits two explanations. The front may be insensitive to quantization error, or the restore may never have read it. Separating the two requires knowing, block by block, what each restore read.

III. Methodology

Both steps share one platform, one harness, and one rule: each experiment changes a single variable. Step 1 changes when

a placement rule runs; Step 2 changes where low-precision blocks sit and holds bytes, codec, and attention path fixed.

A. Platform and harness

All measurements run on one AMD Instinct MI300X (192 GiB HBM3, PCIe Gen5 x16) with ROCm 7.2.2, PyTorch 2.10, and Transformers 5.17. We use Llama-3.1-8B-Instruct for both steps and add Qwen2.5-7B-Instruct-1M [15] for the precision study, all in BF16 weights. We add Qwen as a stress case: KVTuner reports that Qwen2.5-3B and 7B-Instruct degrade sharply when keys drop to 4 or 2 bits [16]. Both models use grouped-query attention. A self-managed KV harness reuses the Hugging Face weights and rotary embedding but owns attention and the KV buffer, so it can time every block and run Cake's two streams concurrently. Recomputing a block twice yields identical bytes, a GPU→pinned→GPU round trip yields identical bytes, and every experiment compares the restored KV against the stored copy; all experiments report zero corrupted blocks.

The harness recomputes 1.7–2.0× slower than default vLLM: a full 32K prefill takes 3.76 s in the harness against 2.24 s for a 33K prefill in vLLM 0.28 with its default attention kernel, and 0.645 s against 0.318 s at 8K. Slower recomputation pushes the meeting point toward the front and leaves a write-time rule less to decide, which biases Step 1 toward our conclusion. § IV-F tests that bias in simulation.

B. Tiers

The CPU tier is pinned host memory with real host-to-device copies (52.9 GiB/s raw H2D). Slow tiers reuse pinned memory behind a rate limiter that charges each block by its actual bytes; we call them "rate-limited tiers" and do not claim they behave like a particular device. Step 1 sets the limiter from the slowest measured value of two real devices times 0.9: a local SSD (6.98 GiB/s read, 1.91 GiB/s write) and an NFS mount (0.331 and 0.601 GiB/s). A real-device calibration sets how much a concurrent write slows reads (3.31× on the SSD, 1.67× on NFS, 1.16× on the CPU tier). The limiter tracks its set rate within 0.3% from 0.25 to 40 GiB/s.

C. Step 1 protocol: the deferral test

Table I lists the policies. A write-time idea passes the deferral test only if, on the GPU and across at least three workload seeds, it beats every deferred twin by at least 5%; we call 15% strong. A rule that beats only some twins counts as a good rule, and its timing earns no credit. The test follows from a simple argument. A write-time decision can win only if it uses information that disappears after the write, saves a resource that deferral must spend, avoids work that deferral can only do in idle time the system lacks, or fixes a format that limits what the reader can do. Token position satisfies none of these on its own.

Step 1 serves eight sessions one request at a time. In the *chat* workload each turn appends 8K tokens for up to four turns; in the *doc* workload the first turn writes 32K tokens and later turns ask short questions. A workload seed sets which sessions return and in what order. The CPU tier holds 25%, 50%, or 100% of the working set. Under *free* release, demotion frees CPU space at once; under *hold*, a request waits until the demotion write completes. The metric is the median TTFT of returning requests.

TABLE I
PLACEMENT POLICIES. ALL POLICIES RESTORE WITH CAKE.

Policy	Decides	What it does
WRITETIME	at write	Blocks before the CPU-speed meeting point go to SSD, the rest to CPU. Variants evict by position, by LRU session, or by a Pensieve-style value [3].
DEFERRED	CPU full	Same rule, applied to the blocks it evicts.
BACKGROUND	after turn	Same rule, run in the background to keep 25% of CPU free.
PRECLEAN	ahead	Copies blocks to SSD in eviction order; keeps the CPU copy.
WRITETHROUGH	no choice	Writes every block to CPU and SSD.
LRU	CPU full	Position-blind: demotes the least recently used session.

TABLE II
PRECISION CONFIGURATIONS AT 32K TOKENS (64 BLOCKS). EVERY MIXED CONFIGURATION STORES 16 LOW-PRECISION BLOCKS; ONLY THEIR POSITION DIFFERS.

Config	Low-precision blocks	Role
BF16	none	accuracy reference
All	0–63	uniform low precision
All+S	1–63	published sink rule
All+SR	1–62	published sink + recent rule
Front	1–16	position: front
Mid	24–39	position: middle
Back	48–63	position: back
Rand	16 random from 1–63	same ratio, no position

A simulator shares the policy code and the rate limiter with the GPU harness and replaces time with a virtual clock. It reproduces all 114 first-phase cells within 10% (median error 0.4%, maximum 5.0%) and 47 of 50 later GPU cells within 10% (median 3.1%). Its error on the gap *between* two policies is larger, about 5–6 percentage points, so we use it only to pick candidates for the GPU and screen at a 10% threshold. A separate concurrency simulator adds 32 sessions, Poisson or bursty arrivals, a GPU queue, an SSD write queue, and drop-on-full admission; no GPU run validates it.

D. Step 2 protocol: read accounting

Configurations. Table II lists the configurations. Every mixed configuration keeps block 0 in BF16, because the sink tokens that published rules protect, the first one to five tokens [17], [10], [11], all fall inside block 0; without that control a front placement would also quantize the sink. Sinks can also emerge beyond the initial tokens [18]; like the published rules, All+S protects only the start. All versus All+S measures the sink on its own. The restore dequantizes every

low-precision block into the same BF16 KV buffer and runs the same attention kernel, so the attention path never changes.

Two read modes. The *full-load* read loads all 64 blocks from storage and measures sensitivity. The *Cake* read recomputes blocks before the measured meeting point and loads the rest, and measures the accuracy a user would get. Because recomputed KV equals the BF16 KV stored at prefill bit for bit, we evaluate the Cake read by replay: blocks before b take the BF16 copy and blocks from b on take the stored copy. On 120 items (50 Llama FP8, 50 Qwen FP8, 20 Qwen INT4, meeting points 15–36) the replay matches a real Cake restore exactly in KV and in logits. We evaluate the Cake read with the GPU idle, with the GPU busy, and on a second turn restored under a busy GPU after an idle first turn.

Tasks. We use RULER-style synthetic tasks [19] with the needed information placed in a chosen position band: multi-value needle retrieval (T2), variable tracking (T3), two-hop QA that inserts the two HotpotQA gold paragraphs [20] into the band (T4), and common-word extraction over the whole context (T5). The bands are blocks 1–16, 24–39, and 48–63, aligned with the mixed configurations so that answer position and low-precision position cross. A multi-key needle task saturates under BF16 on both models, so the decision rules exclude it. Each model answers 600 items (180 each for T2–T4 and 60 for T5); T2–T4 have a second turn, for 1,140 scored rows per condition. We tune difficulty on BF16 only: every model, task, and band sits between 60% and 95% BF16 accuracy before any low-precision run.

Formats. We pick formats on 20 separate items using only the KL divergence of next-token distributions, never accuracy. FP8 is E4M3 [21] with a scale per block and per token (Llama) or per head (Qwen). INT4 follows KIVI, with keys quantized per channel and values per token in groups of 32 [9]. Two 8-bit diagnostics test whether a failure belongs to the format: FP8-K0 subtracts each key channel's mean within a block before FP8 and stores the mean in BF16, and INT8 stores symmetric INT8 with a scale per token and head.

Speed. We pick slow-tier bandwidths by where BF16's meeting point lands, $b^* \in \{16, 32, 48\}$: 5.27, 1.47, and 0.42 GiB/s for Llama and 2.77, 0.76, and 0.22 GiB/s for Qwen. A busy GPU runs a background matrix multiply at 0.7 duty cycle, which slows recomputation 2.07×. Each cell takes three process launches of ten repetitions with configurations shuffled within each repetition.

Decision rules. A configuration meets the quality bound ε if its paired accuracy difference against BF16, bootstrapped over items 10,000 times, has a 95% CI lower bound above -2 points and no task-and-turn point estimate below -5, in all four read conditions. A configuration is not slower if, in every tier and GPU state, its median restore time stays within $(1+\delta^*)$ of the fastest mixed configuration, where $\delta^* = \max(5\%, \text{measured noise})$: 5% for Llama and 9.43% for Qwen. The rules then branch in a fixed order. If uniform low precision restores no faster than BF16 anywhere, precision has nothing to offer and we stop. If All, All+S, or an 8-bit codec change meets both rules, the level needs no position policy and we move one level down, from FP8 to INT4, and stop after that level. At a level where uniform fails, a position placement wins only if it beats Rand by at least 2 points with a CI that excludes zero at $b^* = 32$ and is not slower; otherwise position adds nothing and we stop.

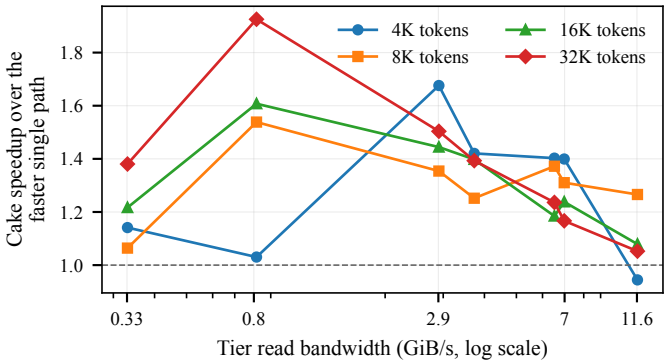


Fig. 2. **Cake pays off between 0.3 and 4 GiB/s.** Speedup of Cake over the faster of recompute-only and load-only, per context length; one run per cell, median of six restores.

E. Measurement hygiene

Run-to-run noise in Step 1 is 6.2% (95% CI half-width over median), mostly from the meeting point jumping between adjacent blocks. A process monitor checks every timed run for foreign GPU processes; one Qwen FP8 speed grid recorded one suspicious sample, and we re-measured the whole grid and judged on the re-measurement. Quality runs execute several processes at once, so we discard their latency fields and keep only their scores. Greedy decoding makes those scores deterministic: on Llama, 2,620 rows that read no low-precision block reproduce the BF16 baseline's text and answer log-probabilities bit for bit across separate batches. We committed the precision study's rules and analysis code before generating any low-precision answer, and the concurrency rerun of § IV-E froze its code by hash before it ran.

IV. Step 1: Placement Can Wait

This section shows that a write-time placement rule never beats the same rule applied later, and it locates the one regime where deferral could lose.

A. Cake pays off in a bandwidth band

Read-adaptive restore is worth the most where loading a block costs about as much as recomputing it (Fig. 2). At 32K tokens Cake beats the faster single path by 1.38–1.93× between 0.33 and 3.73 GiB/s, with the peak at 0.82 GiB/s. Above that band the gain shrinks: 1.17× at 6.98 GiB/s and 1.05× at 11.6 GiB/s. Against recompute-only, longer contexts gain more at all seven bandwidths. One of 28 cells reverses: at 4K tokens and 11.6 GiB/s Cake is 5.6% slower than load-only (78 versus 74 ms). Because our harness recomputes slowly, the band's absolute position would shift to higher bandwidth on a faster engine; its shape is what Step 1 relies on.

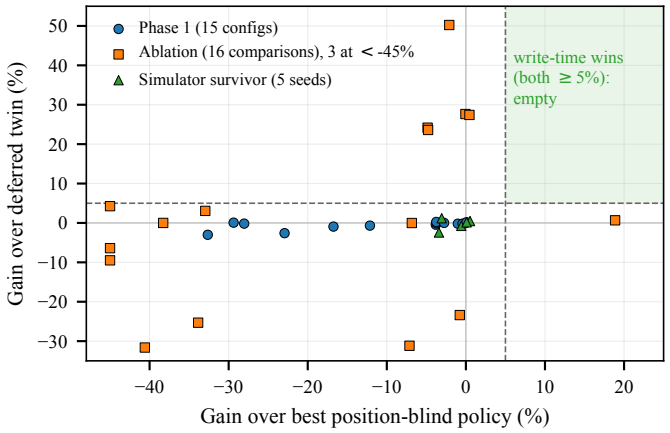


Fig. 3. **The win region is empty.** Each point is one GPU comparison of a write-time rule against the best position-blind policy (x) and against its deferred twin (y).

TABLE III
DEFERRAL TEST ON THE GPU. GAINS ARE FOR THE WRITE-TIME RULE; POSITIVE MEANS FASTER.

Round	n	Pass	Gain range
First phase (chat, free)	15	0	vs. position-blind -32.6 to $+0.1\%$; vs. twin -3.0 to $+0.3\%$
Ablation (hold; slow CPU; tight capacity)	16	0	best: $+18.9\%$ vs. position-blind, $+0.7\%$ vs. twin
Simulator survivor, 5 seeds	5	0	vs. position-blind -3.4 to $+0.5\%$; vs. twin -2.4 to $+1.2\%$
Total	36	0	

The band also bounds what a write-time placement rule can decide. On the CPU tier a block loads 16× faster than the cheapest block recomputes, so the meeting point sits at block 0–3 and a rule that sends "the part Cake will recompute anyway" to SSD has almost nothing to send.

B. No write-time rule beats its deferred twins

Fig. 3 and Table III show the 36 GPU comparisons. The win region, where a write-time rule beats both the best position-blind policy and its deferred twin by 5%, holds no point. Two further first-phase cells under a pessimistic FIFO interference model agree (twin gap -0.1%).

First phase: identical behavior. The write-time rule and its deferred twin differ only in when they apply the position split, and they finish within -3.0% to $+0.3\%$ of each other in all 15 configurations. At CPU speed the computed meeting point covers 0–3 blocks, so the write-time rule places almost every block in CPU memory and then behaves exactly like its twin. In 22 of 22 write-time/twin pairs under free release the two differ by less than 5%. When capacity is tight, position-based eviction itself hurts: at 25% capacity LRU-style policies beat the position rules by 12–33%, because evicting every session's front strands the front of the next returning session on the slow tier.

Ablation: the rule wins, the timing does not. We then combined every condition we expected to favor a write-time rule: a slow CPU tier (3.69 GiB/s, set from vLLM's measured CPU offload throughput on this machine), hold release, the doc workload, and 50% capacity. The best write-time rule beat the best position-blind policy by 18.9% and its deferred twin by only 0.7%. On the next workload seed the same rule lost to write-through by 6.8%.

Simulator survivor: gone on the GPU. Of 64 simulated Llama configurations, one passed the four-twin screen on at least three of five seeds, with a predicted 7.6% gain; four survivors of a second, MHA model could not run on the GPU because the doc workload exceeds that model's 32K-token limit. On the GPU it passed on zero of five seeds; it stayed within $\pm 3.5\%$ of both its deferred twin and the best position-blind policy. The predicted gain was the size of the simulator's relative bias (§ III-C).

C. Write-time decisions go stale

A write-time decision bakes in assumptions about a future read, and the reader often sees a different world. A policy that does not store the front at all and recomputes it on return gains nothing when the user returns after the writes complete (0 of 24 cells). When the user returns before the writes finish, it is faster in 6 of 24 cells (by 3.7–13%), and only 3 of those exceed the 6.2% run-to-run noise. When the GPU is busy, the same policy runs 13–66% slower than storing every block at 16K tokens and beyond, because a busy GPU should shift the meeting point forward and the missing front blocks forbid that shift. The position split itself carries the same risk: computed for an idle GPU, it makes the write-time rule 37% slower than keeping everything in CPU memory on the NFS tier with a busy GPU at 32K (546 versus 400 ms).

D. Return order picks the winner

The best policy depends on which sessions return and in what order more than on hardware constants. In simulation, keeping the hardware fixed and changing only the workload seed leaves the best non-write-time policy unchanged across five seeds in 17 of 64 Llama configurations. On the GPU, three seeds of the same first-phase configuration spread the policies by 29.5%, 32.6%, and 0.1%, and five seeds of the simulator survivor produced four different winners. A rule that predicts the winner from the recompute-to-load cost ratio therefore cannot work for the write side; such a rule does predict Cake's read-side gain (median error 4.1%).

E. Deferral needs idle time, and one GPU leaves enough

A deferred twin needs time to move blocks after the fact, so it can lose only when the SSD write path never drains. We define write load ρ_{KV} as the rate at which the system wants to write KV to SSD divided by the SSD's write bandwidth. Across 34,560 concurrency simulations on one GPU (0.05–0.4 sessions/s, five seeds), ρ_{KV} never exceeds 0.99, and its peak on the local SSD is 0.34.

The pre-registered analysis first reported a winner in 5 of 192 cells. Two modeling errors in the twins explained all five:

TABLE IV
SIGNALS AVAILABLE AT WRITE TIME AND THEIR DEFERRED COUNTERPARTS.

Signal	Deferred counterpart	Evidence
Token position	Same rule at eviction, new blocks eligible	36 GPU comparisons; fresh-seed rerun
Future reuse	Same oracle applied at eviction; timing matters only when CPU holds $\leq$10% of the working set	simulation on Mooncake traces [4]
Precision	Keep BF16 in CPU, quantize at demotion; same final layout	Step 2
Idle time	Present unless $\rho_{\mathrm{KV}} \geq 1$ with a bounded queue	simulation; vLLM; page cache

a twin labeled "deferred" evicted in a different order than its write-time rule, and the drop model never let a policy evict a newly arriving block, unlike the GPU harness. With both fixed, no cell passed. Because we fixed them after seeing the winners, we re-registered the corrected setup, froze the code by hash, and reran it on ten fresh seeds. Among the 216 cells without sustained backlog ($\rho_{\mathrm{KV}} < 1$), neither write-time candidate passed in any cell, under the median, the mean, or the 90th percentile.

Write-time placement survives only in an overloaded corner that we had to assume: eight GPUs sharing one local SSD (0.239 GiB/s write each, not measured), a bounded write queue of 16, and $\rho_{\mathrm{KV}} \approx 2.1$. There, one of five earlier winning cells reproduced on 7 of 10 fresh seeds with a median gain of 14.2% at a median returning TTFT of 11.2 s, and one new cell passed on 6 of 10 seeds at 12–32 s. Under the mean or the 90th percentile, none of these cells wins.

Real systems confirm that a single GPU does not starve deferral. vLLM 0.28 with a CPU tier at 50% of the working set skipped 0 of 133,504 blocks offered for offload, and 0 of 200,256 at 25%: when a store fails, its scheduler retries later. The operating system adds another deferral buffer: files written with buffered I/O remain 100% in the page cache 30 s after the write, on the local SSD and on NFS.

F. Faster recomputation does not revive write-time placement

Because our harness recomputes slowly (§ III-A), we reran the simulator with recomputation $2\times$ and $1.67\times$ faster against the full twin set. Under free release no configuration survived at either speed (0 of 32; largest lead 4.9%). Under hold release one configuration survived the 10% screen, but it split by seed: it won by 22–30% on three seeds and lost to write-through by 20–27% on two. No GPU run confirms these simulations.

G. Every write-time signal we tried is available later

Position is one of several signals a system knows at write time, and Table IV shows that deferral recovers each of the ones we tested. Future reuse is the most tempting: in the Mooncake conversation and tool-agent traces, 75.8% and 78.5% of blocks are never read again. In simulation, applying an oracle's reuse knowledge at eviction instead of at write gives identical TTFT and SSD write volume on the 11.6 GiB/s CPU tier with a local SSD and on every NFS cell; elsewhere the differences come from the rule, except when the CPU tier holds at most 10% of the working set, which turns the question into one of admission. Write volume offers no refuge either: a write-time rule that stays within 5% of the best TTFT while writing at least 20% less to SSD appears in only 2 of 23 settings, and plain deferral writes the least in 10 of 23, because it writes only what overflows.

Takeaway. Token position is permanent, so deferral sees everything a write-time placement rule sees, plus the GPU load and bandwidth at the moment of the move. On one GPU the deferred twin always has time to act, and every apparent write-time win in our data came from the rule or from a modeling error.

TABLE V
RESTORE-TIME REDUCTION OF UNIFORM LOW PRECISION (ALL) AGAINST BF16, 32K TOKENS, MEDIANS. NEGATIVE MEANS SLOWER. BANDWIDTHS PUT BF16'S MEETING POINT AT BLOCK 16, 32, OR 48.

		Llama-3.1-8B		Qwen2.5-7B	
Tier	GPU	FP8	INT4	FP8	INT4
b^*=16	idle	34.6%	52.0%	35.6%	47.5%
	busy	24.7%	36.1%	39.2%	41.6%
b^*=32	idle	31.1%	50.9%	35.3%	49.0%
	busy	37.2%	55.2%	38.0%	56.9%
b^*=48	idle	22.7%	37.6%	22.0%	38.1%
	busy	28.3%	47.5%	30.4%	48.3%
CPU (real H2D)	idle	24.8%	17.8%[a]	−5.1%	−17.9%
	busy	−3.5%	−15.4%	−12.6%	−121.4%

[a] Paired 95% CI includes zero.

TABLE VI
WHERE THE TIME GOES ON THE REAL CPU TIER (MS, MEDIANS, 64 BLOCKS).

Model	GPU	BF16 H2D	FP8 H2D / deq.	INT4 H2D / deq.	Restore BF16→FP8→INT4
Llama	idle	79.8	39.5 / 24.0	27.2 / 44.2	95.5→71.7→78.5
Llama	busy	116.4	43.8 / 64.0	29.4 / 112.6	164.1→170.8→189.4
Qwen	idle	36.5	17.7 / 10.7	14.4 / 25.5	51.8→54.9→61.0
Qwen	busy	50.1	19.8 / 26.1	15.7 / 64.3	65.9→74.1→145.9

V. STEP 2: PRECISION DOES NOT NEED POSITION

Precision looks like the strongest case for deciding early: a block stored in FP8 cannot return to BF16 without a second copy. This section shows that the case fails under Cake. Position mostly controls whether the restore reads a low-precision copy at all, uniform formats meet the quality bound once the codec fits the model, and a fixed ratio placed by position does no better than the same ratio placed at random.

A. Low precision speeds slow tiers and slows the CPU tier

Low precision has to pay for itself in restore time before position can matter, and it does so only on slow tiers (Table V). On rate-limited tiers uniform FP8 cuts restore time by 22–39% and INT4 by 36–57%, on both models and with the GPU

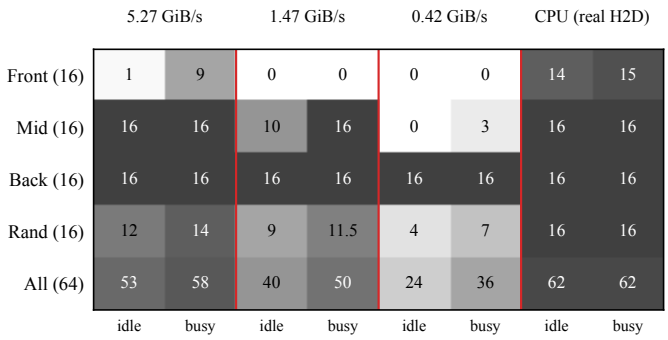


Fig. 4. **Cake skips low-precision front blocks on slow tiers and reads them on the CPU tier.** Low-precision blocks read per restore (of 16 stored; 64 for All), Llama FP8, median. Qwen matches within one block.

idle or busy. Fewer bytes also pull the meeting point back: at $b^*=32$ with an idle GPU, FP8 moves Llama's meeting point from block 31–32 to 22–24. On the real CPU tier the gain vanishes. FP8 helps only Llama with an idle GPU, and INT4 slows Qwen with a busy GPU by 2.2×.

Table VI explains the reversal. On the CPU tier the copy is short (36.5 ms for all 64 Qwen BF16 blocks), so halving or quartering the bytes saves 19–87 ms at most. Dequantization runs on the GPU, where it competes with recomputation; under load it grows from 10.7 to 26.1 ms for Qwen FP8 and to 64.3 ms for INT4, which eats the saving. Moving from FP8 to INT4 saves only 3–14 ms more of copying but multiplies dequantization time by 1.8–2.5. Our dequantization uses one PyTorch kernel per step; a fused kernel produces bit-identical KV, and its CPU-tier timing is **[PENDING:** fused-dequantization speed grid, stage 42].

B. Front low-precision copies are mostly never read

Read accounting shows that the stored position mostly decides whether the restore reads a low-precision copy (Fig. 4). On rate-limited tiers with an idle GPU, Cake reads 0–1 of the 16 front copies, because they lie before the meeting point and come back as exact BF16, and it reads all 16 back copies. A busy GPU recomputes less, so the meeting point moves forward (from block 31–32 to 20–21 at $b^*=32$), and at $b^*=16$ the restore starts to read front copies (9 of 16). On the real CPU tier the meeting point lies at block 1–3, so Cake reads 14–16 of the 16 front copies, like every other placement. Qwen matches these counts within one block, because we chose its bandwidths to put the same meeting points in the same places.

The Cake read alone therefore cannot measure the sensitivity of the front. At the balanced point ($b^*=32$) Front reads no low-precision block, so its accuracy equals BF16 by construction. We judge sensitivity on the full-load read and report the Cake read as the accuracy a user would see.

C. Uniform FP8 suffices for Llama; Qwen fails on its key format

Table VII reports the uniform configurations; the fourth condition (idle first turn, busy second turn) agrees with the other three in every row. For Llama, uniform FP8 meets the bound in all four conditions, with its worst difference at -0.5 points and 82–86% of outputs identical to BF16. FP8 is therefore enough for Llama, and the FP8 level needs no position policy.

Qwen loses 32.8 points under the full load. The loss concentrates in tasks that aggregate information across the context: two-hop QA drops 71.7 and 67.2 points on its two turns and common-word extraction 51.5, against 16–18 for multi-value retrieval and 8–10 for variable tracking. The Cake read loses less (-25.4 idle) because it recomputes the 23 blocks before the meeting point in BF16, which agrees with § V-B. Protecting the sink (All+S) or the sink and the most recent block (All+SR) recovers 0.3–0.5 points.

Changing the codec at the same 8 bits recovers everything. Subtracting each key channel's mean before FP8 (FP8-K0) turns -32.8 into $+0.4$ points and cuts KL from 3.84 to 0.0049; INT8 with a per-token scale gives $+0.2$. We read the failure as a format mismatch: Qwen's keys carry a large constant offset per channel, which stretches FP8's per-head scale and leaves its 3 mantissa bits too coarse for the small differences that attention uses [21]. FP8-K0 misses the speed rule on the busy CPU tier (16.4% slower than the fastest mixed configuration, against a 9.43% allowance) because adding the mean back costs an extra dequantization step, so INT8 is the 8-bit format that passes both rules.

D. Uniform INT4 keeps quality on both models

Since the FP8 level needed no position policy on either model, the rules move one level down to INT4 (KIVI layout). Uniform INT4 meets the quality bound on both models in all 24 model, configuration, and condition cells; the lowest CI bound is -0.8 points. Qwen, which lost 32.8 points under FP8, loses nothing under INT4. KIVI quantizes keys per channel with a per-channel zero point, which removes the same constant offset that FP8-K0 removes, so we attribute both recoveries to one cause. One cell, Llama All under the full load, shows a CI that excludes zero on the positive side ($+1.1$); among 24 cells we expect about one such cell by chance, and we do not read it as INT4 beating BF16.

INT4 does leave a mark below the decision threshold. Its outputs match BF16 verbatim in 74.6% (Llama) and 73.9% (Qwen) of rows, against 82–91% for the 8-bit formats, and the log-probability of the reference answer drops slightly (Qwen common-word extraction: -0.0275 ± 0.0051 per token). At 32K tokens these shifts do not flip answers; whether they do at 64K and 128K is **[PENDING:** pre-registered long-context study, 64K and 128K].

Uniform INT4 fails only the speed rule, and only on the real CPU tier (Llama busy: 1.13–1.15× the fastest mixed configuration against 1.05; Qwen: 1.14× idle and 2.10× busy against 1.094). By the frozen rules that failure sends INT4 to the position test, which we run on Llama, the model the rules select when both models fail.

E. Position does not beat random placement

With 16 of 64 blocks in INT4, placing them by position buys nothing over placing them at random (Fig. 5). Against

TABLE VII

UNIFORM LOW PRECISION AGAINST BF16: PAIRED ACCURACY DIFFERENCE IN POINTS WITH 95% CI, OVER 1,140 ROWS (T2–T5, BOTH TURNS). THE BOUND IS A CI LOWER BOUND ABOVE −2 IN ALL FOUR CONDITIONS. KL IS THE MEAN NEXT-TOKEN KL DIVERGENCE UNDER THE FULL LOAD. "NOT SLOWER" APPLIES THE SPEED RULE IN EVERY TIER.

Model	Format	Config	Full load	Cake, idle	Cake, busy	KL	Bound	Not slower
Llama	FP8	All	−0.5 [−1.1, +0.2]	−0.2 [−0.8, +0.3]	−0.5 [−1.1, +0.1]	0.0035	pass	pass
		All+S	−0.3 [−1.0, +0.4]	−0.2 [−0.8, +0.3]	−0.5 [−1.1, +0.1]	0.0035	pass	pass
		All+SR	−0.5 [−1.2, +0.1]	−0.4 [−1.0, +0.2]	−0.3 [−0.9, +0.2]	0.0033	pass	pass
Qwen	FP8	All	−32.8 [−35.8, −30.0]	−25.4 [−28.5, −22.4]	−27.7 [−30.7, −24.7]	3.84	fail	fail (CPU busy)
		All+S	−32.5 [−35.5, −29.6]	−25.4 [−28.5, −22.4]	−27.7 [−30.7, −24.7]	3.83	fail	pass
		All+SR	−32.3 [−35.3, −29.5]	−24.3 [−27.4, −21.3]	−26.4 [−29.4, −23.4]	3.63	fail	fail (CPU busy)
	FP8-K0 INT8	All+S	+0.4 [−0.3, +1.1]	+0.2 [−0.5, +1.0]	+0.3 [−0.4, +1.0]	0.0049	pass	fail (CPU busy)
		All+S	+0.2 [−0.5, +0.9]	+0.2 [−0.4, +0.8]	−0.3 [−1.0, +0.4]	0.0083	pass	pass
Llama	INT4	All	+1.1 [+0.3, +1.9]	+0.3 [−0.4, +0.9]	+0.3 [−0.3, +1.0]	0.0068	pass	fail (CPU busy)
		All+S	+0.2 [−0.5, +0.9]	+0.3 [−0.4, +0.9]	+0.3 [−0.3, +1.0]	0.0054	pass	fail (CPU busy)
		All+SR	+0.3 [−0.4, +1.0]	+0.1 [−0.6, +0.8]	+0.0 [−0.7, +0.7]	0.0051	pass	fail (CPU busy)
Qwen	INT4	All	+0.1 [−0.8, +0.9]	+0.7 [−0.0, +1.4]	+0.4 [−0.3, +1.1]	0.0134	pass	fail (CPU)
		All+S	+0.4 [−0.4, +1.2]	+0.7 [−0.0, +1.4]	+0.4 [−0.3, +1.1]	0.0098	pass	fail (CPU)
		All+SR	+0.0 [−0.8, +0.9]	+0.4 [−0.3, +1.2]	+0.3 [−0.4, +1.0]	0.0098	pass	fail (CPU)

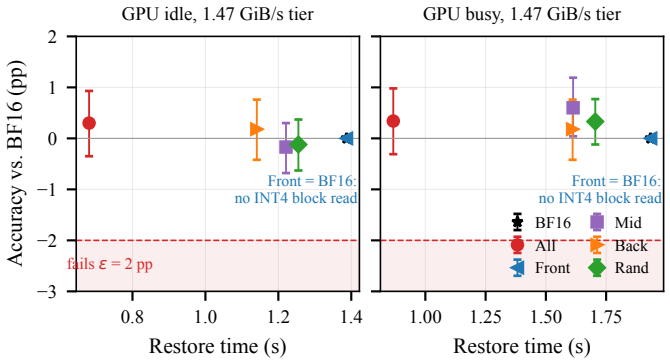


Fig. 5. **Placing 25% INT4 by position matches random placement; uniform INT4 is fastest at equal accuracy.** Llama, Cake read at 1.47 GiB/s (b^*=32); 95% CI over items.

Rand on the Cake read at b^*=32, Front, Mid, and Back differ by +0.12, −0.06, and +0.30 points with an idle GPU and by −0.33, +0.27, and −0.15 with a busy one. Every CI contains zero, and every difference lies far inside the 2-point margin. Under the full load all four placements stay within +0.2 to +0.6 points of BF16. Even when the INT4 blocks cover exactly the band that holds the answer, accuracy does not drop (+0.53 to +0.65 points on T2–T4; exploratory, no CI test). Position does change speed: Back restores 9% faster than Rand with an idle GPU, because its INT4 blocks sit where Cake loads. Uniform INT4 is faster still (0.68 versus 1.14 s) at the same accuracy, so a system that wants to save time should lower precision everywhere and not choose where. The same comparison for FP8 on both models and for INT4 on Qwen is **[PENDING:** report-only mixed placements, stages 45 and 46].

Takeaway. Under read-adaptive restore, a block's position mostly predicts whether the restore reads its low-precision copy; the codec decides how much the copy hurts. Once the codec fits the model's key distribution, uniform low precision meets the quality bound at both levels we tested, and a fixed ratio placed by position matches the same ratio placed at random.

VI. DISCUSSION

Both steps point to one design, and our data leave a few places where an early decision could still pay.

A. *Write simply, demote lazily, decide at read time*

Both parts point to the same design. Write every block to the CPU tier in BF16. When the CPU tier fills, demote blocks to the slow tier and quantize them on the way, choosing which blocks by recency. At read time, let the restore split recomputation from loading according to the GPU load and bandwidth it observes. This order never writes more bytes to the slow tier than a write-time rule would, because the slow tier receives the same low-precision copy; it costs CPU space until demotion, and a user who returns early gets BF16.

B. *Where an early decision could still pay*

Our data leave four openings. First, a sustained SSD write backlog with a bounded queue: the one reproduced simulated cell needs eight GPUs per SSD, a regime we did not measure, and gains only on the median. Second, information that disappears after the write: hybrid attention–SSM models overwrite intermediate states during prefill, so a cache must choose checkpoints while it prefills [22]; this choice depends on content. Third, a write path open only at write time, such as GPU-direct writes to an SSD that bypass host memory; we could not measure it on our platform. Fourth, admission when the CPU tier is tiny (at most 10% of the working set), where knowing reuse in advance avoids SSD writes altogether.

C. *On a fast tier, low precision buys capacity*

On the real CPU tier low precision saves little time (§ V-A), but it multiplies capacity. A 32K-token session takes 4 GiB in BF16 and 1.25 GiB in INT4 (KIVI, including scales) for Llama, and 1.75 GiB and 0.55 GiB for Qwen. One TiB of host

memory therefore holds 256 Llama sessions in BF16 and 819 in INT4, and 585 and 1,872 for Qwen, 3.2× more in both cases. Capacity decides when sessions spill to the slow tier, where low precision does save time; a measured two-tier restore (CPU in BF16, slow tier in INT4) is **[PENDING:** two-tier restore grid, stage 44**]**.

D. Pitfalls we hit

Four measurement errors would each have produced a false positive for early decisions. *Unread copies.* A Cake read at the balanced point reads no front copy (§ V-B); reporting only that read would call the front insensitive. *Post-hoc rivals.* We added several twins after seeing a write-time winner; a fresh-seed, hash-frozen rerun of the corrected setup (§ IV-E) is what lets us keep the conclusion. *Simulator bias.* The simulator predicts absolute TTFT within a few percent but the gap between two policies with an error near 5 points, the size of the one survivor's predicted gain. *Kernel swaps.* vLLM 0.28 on ROCm drops its default ROCM_ATTN kernel whenever a deployment configures any KV connector, because that kernel's KV layout is not blocks-first [23]; an open issue reports the same exclusion for disaggregated decoding [24]. On our machine the engine fell back to a Triton kernel. With CPU offload enabled, new 33K-token requests ran 96% slower on 3 of 3 seeds; the kernel swap alone explained most of that, attention time for one prefill rose from 1.37 to 3.28 s, and registering K and V as separate views restored the default kernel and removed 45–74% of the extra time. An early version of our own comparison had judged the harness fast by comparing it against this slowed vLLM.

E. Energy changes the read-side answer

Measured in GPU energy instead of time, Cake loses its advantage on fast tiers. The MI300X draws 726.4 W while recomputing and 168.9 W while waiting for I/O, so load-only restore uses 62–70% less GPU energy than Cake on the CPU and local-SSD tiers at a cost of 4–43% more restore time; on the NFS tier Cake wins both. This result comes from one seed and covers only the GPU board.

VII. Related Work

Prior KV systems decide either at read time or at write time, and few compare a write-time decision against the same decision made later.

Read-adaptive restore. Cake meets recomputation and loading in the middle and assumes that the system computed and stored the full KV beforehand [13]. CacheGen chooses a compression level per chunk when it streams the chunk and can fall back to sending text [14]. KVPR overlaps partial recomputation with transfer [25], CacheFlow plans recomputation and loading at restore time [26], and HCache restores from hidden states, splitting the work by layer [27]. Pensieve swaps KV from GPU to CPU memory and, when CPU memory fills, drops the chunks with the lowest ratio of recomputation cost to idle time, which favors the leading tokens of idle conversations; it recomputes dropped tokens on return [3]. It is a deferred form of "do not keep the front." These systems decide at read time or on eviction; we show that write time adds nothing to them.

Write-time placement and admission. Krul [6], TierKV [7], and Lachesis [8] decide placement when they write, EfficientAgent decides admission to the host tier [28], and a CXL memory rack skips writes when its buffer fills [29]. AdaptCache [30] and EvicPress [31] choose a lossy compression level and a tier for each entry when they store it and revisit the choice as tiers fill, and KVDrive persists all KV to SSD at prefill and keeps attention-selected entries in HBM and DRAM [32]. Serving engines and KV layers such as vLLM [33], SGLang [34], LMCache [5], and Mooncake [4] keep KV in CPU memory or storage for reuse across requests. A recent study of placement policies across GPU, CPU, and SSD finds that tier capacity, more than policy, drives the gains [35]. Few of these designs compare a write-time rule against the same rule applied later; our deferral test makes that comparison.

Position-aware and content-aware KV precision. KIVI keeps a full-precision residual of recent tokens [9], KVQuant keeps the first token in fp16 [10], SKVQ keeps a recent window and five sink tokens [11], and QEvict [12] and PM-KVQ [36] combine both rules; all of them quantize KV inside GPU memory at low bit widths. StreamingLLM identifies the attention sink under eviction [17], and ZipCache corrects the bias of accumulated attention toward early tokens [37]. Content-aware schemes choose precision from attention, similarity, or internal metrics [38], [39], [40], and KVTuner tunes precision per layer [16]. PatchKV, the closest work, recomputes a predicted set of blocks and quantizes only the blocks it will load, with per-block precision frozen at offload from an attention-weighted risk score [41], and SparKV evaluates Cake with a quantized loaded segment as a baseline [42]. Our read accounting adds what these evaluations lack: a per-block record of which stored copies a read-adaptive restore actually reads.

Multi-level caching and recomputable storage. Gill compared promoting data to a cache level as it passes against demoting it on eviction and found small differences when demotion is free [43], and DEMOTE, which returns blocks to the lower cache on eviction, approximates exclusive caching when the lower cache also discards the blocks it just sent up [44]. Tachyon uses lineage to checkpoint in the background without stalling writes, and it checkpoints synchronously once memory fills with files it has not yet persisted [45]; a lack of idle time is likewise the only regime where our deferred twins lose. TPP separates reclamation from allocation watermarks so that background demotion keeps headroom for bursts [46]. Our results extend this line to KV caches, which a system can recompute and which an adaptive restore reads.

Characterizing offload. A recent analysis derives the ratio of cached to new tokens beyond which offloaded prefill becomes transfer-bound, and measures on H100 that transfers take up to 99% of a long request's latency [47]. On an H200, offloading under high KV pressure saves 14.4–31.9% of idle-subtracted GPU energy per output token compared with recomputation after preemption [48]; our energy measurement (§ VI) instead compares restore paths for one returning request. We study a

different axis: the timing of placement and precision decisions, given an adaptive reader.

VIII. Scope and Threats to Validity

One platform. Every measurement comes from one MI300X; other GPUs change recompute speed and therefore the meeting point. **Slow recomputation.** The harness recomputes 1.7–2× slower than default vLLM; faster recomputation moves the Cake band and the meeting points, and only simulation covers it (§ IV-F). **Rate-limited tiers.** Slow tiers are pinned memory behind a limiter; device effects such as write cliffs and file-system overhead are absent, and we name tiers by bandwidth. **Single-request GPU harness.** Concurrency appears only in simulation without GPU validation, and per-GPU SSD bandwidth when several GPUs offload at once is not measured. **Models and lengths.** Placement uses one GQA model on the GPU; precision uses two GQA models at 32K tokens with synthetic tasks; MHA models, longer contexts, and real conversations remain open ([**PENDING:** 64K/128K study; LongBench subset]). **Small samples where noted.** The vLLM drop count and the energy result each come from one seed.

IX. Conclusion

Read-adaptive restore changes when a KV cache manager needs to decide. Across 36 GPU comparisons, write-time placement never beat the same rule applied later, and under Cake the stored position of a low-precision block mostly decides whether the restore reads it, while the codec decides how much it hurts accuracy. Uniform formats that fit the model's key distribution meet a 2-point bound at FP8 and INT4, and position adds nothing to a fixed ratio, so a system can write simply, demote and quantize lazily, and decide at read time.

References

[1] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian *et al.*, "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[2] B. Gao, Z. He, P. Sharma, Q. Kang, D. Jevdjic, J. Deng, X. Yang, Z. Yu, and P. Zuo, "Cost-Efficient large language model serving for multi-turn conversations with CachedAttention," in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. Santa Clara, CA: USENIX Association, 2024, pp. 111–126. [Online]. Available: https://www.usenix.org/conference/atc24/presentation/gao-bin-cost

[3] L. Yu, J. Lin, and J. Li, "Stateful large language model serving with Pensieve," in *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. Rotterdam, Netherlands: ACM, 2025, pp. 144–158.

[4] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot," in *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. Santa Clara, CA: USENIX Association, 2025, pp. 155–170. [Online]. Available: https://www.usenix.org/conference/fast25/presentation/qin

[5] Y. Liu, Y. Cheng, J. Yao, Y. An, X. Chen, S. Feng, Y. Huang, S. Shen, R. Zhang, K. Du, and J. Jiang, "LMCache: An efficient KV cache layer for enterprise-scale LLM inference," arXiv:2510.09665, 2025.

[6] J. Wen, J. Liang, Z. Hong, W. Chen, T. Cai, and Z. Zheng, "Krul: Efficient state restoration for multi-turn conversations with dynamic cross-layer KV sharing," arXiv:2507.08045, 2025.

[7] Z. Shu, M. M. R. Sanim, J. Hu, K. Yuan, M. Qin, G. Agrawal, and W. Niu, "TierKV: Long-context on-device LLMs via predictive multi-tier KV caching," arXiv:2609.21172, 2026.

[8] J. Yang, J. Lee, H. Park, S. Y. Lee, N. S. Kim, and J. W. Lee, "Lachesis: Lifetime-aware KV cache placement for agent serving across HBM and high-bandwidth flash," arXiv:2610.08378, 2026.

[9] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "KIVI: A tuning-free asymmetric 2bit quantization for KV cache," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 32 332–32 344. [Online]. Available: https://proceedings.mlr.press/v235/liu24bz.html

[10] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "KVQuant: Towards 10 million context length LLM inference with KV cache quantization," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*. Curran Associates, Inc., 2024, pp. 1270–1303.

[11] H. Duanmu, Z. Yuan, X. Li, J. Duan, X. Zhang, and D. Lin, "SKVQ: Sliding-window key and value cache quantization for large language models," in *First Conference on Language Modeling (COLM)*, 2024. [Online]. Available: https://openreview.net/forum?id=nI6JyFSnyV

[12] A. Garg, A. Gupta, S. Bhattacharya, A. Gupta, S. Kumar, and M. Kumar, "QEvict: Recoverable quantized KV eviction for attention-drift-robust long-context decoding," arXiv:2608.05326, 2026.

[13] S. Jin, X. Liu, Q. Zhang, and Z. Mao, "Compute or load KV cache? why not both?" in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 28 031–28 043. [Online]. Available: https://proceedings.mlr.press/v267/jin25d.html

[14] Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, and J. Jiang, "CacheGen: KV cache compression and streaming for fast large language model serving," in *Proceedings of the ACM SIGCOMM 2024 Conference*. Sydney, NSW, Australia: ACM, 2024, pp. 38–56.

[15] A. Yang, B. Yu, C. Li, D. Liu, F. Huang, H. Huang, J. Jiang, J. Tu, J. Zhang, J. Zhou, J. Lin, K. Dang, K. Yang, L. Yu, M. Li, M. Sun, Q. Zhu, R. Men, T. He, W. Xu, W. Yin, W. Yu, X. Qiu, X. Ren, X. Yang, Y. Li, Z. Xu, and Z. Zhang, "Qwen2.5-1M technical report," arXiv:2501.15383, 2025.

[16] X. Li, Z. Xing, Y. Li, L. Qu, H.-L. Zhen, Y. Yao, W. Liu, S. J. Pan, and M. Yuan, "KVTuner: Sensitivity-aware layer-wise mixed-precision KV cache quantization for efficient and nearly lossless LLM inference," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 36 451–36 485. [Online]. Available: https://proceedings.mlr.press/v267/li25dd.html

[17] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://openreview.net/forum?id=NG7sS51zVF

[18] Z. Su and K. Yuan, "KVSink: Understanding and enhancing the preservation of attention sinks in KV cache quantization for LLMs," in *Second Conference on Language Modeling (COLM)*, 2025. [Online]. Available: https://openreview.net/forum?id=gIqb6zWZoO

[19] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg, "RULER: What's the real context size of your long-context language models?" in *First Conference on Language Modeling (COLM)*, 2024. [Online]. Available: https://openreview.net/forum?id=kIoBbc76Sy

[20] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Brussels, Belgium: Association for Computational Linguistics, 2018, pp. 2369–2380.

[21] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellempudi, S. Oberman, M. Shoeybi, M. Siu, and H. Wu, "FP8 formats for deep learning," arXiv:2209.05433, 2022.

[22] R. Pan, Z. Wang, Z. Jia, C. Karakus, L. Zancato, T. Dao, Y. Wang, and R. Netravali, "Marconi: Prefix caching for the era of hybrid LLMs," in *Proceedings of Machine Learning and Systems (MLSys 2025)*, M. Zaharia, G. Joshi, and Y. Lin, Eds., vol. 7. Santa Clara, CA, USA: MLSys, 2025. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2025/file/7c180af017258d239bac6248d1eb26ac-Paper-Conference.pdf

[23] NickLucche, "[Attention][AMD] standardize kv layout to blocks first for AMD," vllm-project/vllm, pull request #43660, May 2026, GitHub

pull request, merged 2026-05-28 (UTC), merge commit 5b115bb8a3. [Online]. Available: https://github.com/vllm-project/vllm/pull/43660

[24] chunfangamd, "[Performance][ROCm] KV connectors rule out ROCM_ATTN, so PD workers decode on ROCM_AITER_UNIFIED_ATTN (1.9–3.2x the per-token time for Qwen3 models on MI355X)," vllm-project/vllm, issue #60316, Oct. 2026, GitHub issue, opened 2026-10-06 (UTC); still open on 2026-10-16. [Online]. Available: https://github.com/vllm-project/vllm/issues/60316

[25] C. Jiang, L. Gao, H. E. Zarch, and M. Annavaram, "KVPR: Efficient LLM inference with I/O-aware KV cache partial recomputation," in *Findings of the Association for Computational Linguistics: ACL 2025*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, 2025, pp. 19 474–19 488. [Online]. Available: https://aclanthology.org/2025.findings-acl.997/

[26] S. Nian, H. Shen, Z. Wu, J. Fang, Q. Feng, and F. Lai, "CacheFlow: Efficient LLM serving via automated 3D-parallel KV cache restoration," arXiv:2604.25080, 2026.

[27] S. Gao, Y. Chen, and J. Shu, "Fast state restoration in LLM serving with HCache," in *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. Rotterdam, Netherlands: ACM, 2025, pp. 128–143.

[28] K. Shao, J. Chen, J. Yu, X.-H. Li, C. Tao, Y. Wang, H. Lin, K.-T. Cheng, C. Y. Tsui, and H. Bai, "EfficientAgent: What makes KV cache offloading work for concurrent agents?" arXiv:2609.33762, 2026.

[29] H. Jang, I. Song, H. Kim, S. H. Noh, and J. Kim, "A CXL memory rack for multi-turn LLM serving," arXiv:2607.18141, 2026.

[30] S. Feng, H. Li, K. Du, Z. Gu, Y. Liu, J. Yao, S. Ray, S. Shen, Y. Cheng, G. Ananthanarayanan, and J. Jiang, "AdaptCache: KV cache native storage hierarchy for low-delay and high-quality language model serving," arXiv:2509.00105, 2025, 1st Workshop on Big Memory (BigMem '25), co-located with SOSP 2025.

[31] S. Feng, Y. Liu, H. Li, X. Chen, S. Shen, K. Du, Z. Gu, R. Zhang, Y. Huang, Y. Cheng, J. Yao, Q. Zhang, G. Ananthanarayanan, and J. Jiang, "EVICPRESS: Joint KV-cache compression and eviction for efficient LLM serving," arXiv:2512.14946, 2025.

[32] J. Lin, J. Mi, Z. Hong, H. Wang, Q. Liu, H. Zhang, P. Li, and S. Guo, "KVDrive: A holistic multi-tier KV cache management system for long-context LLM inference," arXiv:2605.18071, 2026.

[33] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. Koblenz, Germany: ACM, 2023, pp. 611–626.

[34] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 62 557–62 583.

[35] Srikanta Datta Tumkur, J. Iyer, M. Simhadri, Sai Pavan Kumar, Sai Kapil Kumar, and R. Nampelly, "Where should the KV cache live? Placement policies across GPU, CPU, and SSD for long-lived sessions," arXiv:2609.16215, 2026.

[36] T. Liu, S. Li, J. Yang, T. Zhao, F. Zhou, X. Song, G. Dai, S. Yan, H. Yang, and Y. Wang, "PM-KVQ: Progressive mixed-precision KV cache quantization for long-CoT LLMs," in *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. [Online]. Available: https://openreview.net/forum?id=Vem6FQvRvq

[37] Y. He, L. Zhang, W. Wu, J. Liu, H. Zhou, and B. Zhuang, "ZipCache: Accurate and efficient KV cache quantization with salient token identification," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*. Curran Associates, Inc., 2024, pp. 68 287–68 307.

[38] Y. Zhang, Y. Hu, R. Zhao, J. C. S. Lui, and H. Chen, "DiffKV: Differentiated memory management for large language models with parallel KV compaction," in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*. ACM, 2025, pp. 431–445.

[39] W. Tao, B. Zhang, X. Qu, J. Wan, and J. Wang, "Cocktail: Chunk-adaptive mixed-precision quantization for long-context LLM inference," in *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 2025, pp. 1–7.

[40] D. Lee, D.-H. Kim, and J.-H. Kim, "SemKV: Semantic mixed-precision KV cache quantization guided by the quality cliff for long-context LLM inference," arXiv:2608.28911, 2026.

[41] G. Yang, R. Guo, S. He, S. Chen, Y. Hu, and K. Li, "PatchKV: Efficient KV cache recovery for dynamically edited LLM contexts," arXiv:2609.26219, 2026.

[42] H. Liu, L. Zhai, J. Wang, Z. Fang, J. Chen, and J. Huang, "SparKV: Overhead-aware KV cache loading for efficient on-device LLM inference," *IEEE Internet of Things Journal*, vol. 13, no. 14, pp. 30 016–30 027, 2026.

[43] B. S. Gill, "On multi-level exclusive caching: Offline optimality and why promotions are better than demotions," in *Proceedings of the 6th USENIX Conference on File and Storage Technologies (FAST '08)*. San Jose, CA, USA: USENIX Association, Feb. 2008, pp. 49–65. [Online]. Available: https://www.usenix.org/legacy/event/fast08/tech/full_papers/gill/gill.pdf

[44] T. M. Wong and J. Wilkes, "My cache or yours? making storage more exclusive," in *Proceedings of the 2002 USENIX Annual Technical Conference, General Track (USENIX ATC '02)*. Monterey, CA, USA: USENIX Association, Jun. 2002, pp. 161–175. [Online]. Available: https://www.usenix.org/legacy/publications/library/proceedings/usenix02/full_papers/wong/wong.pdf

[45] H. Li, A. Ghodsi, M. Zaharia, S. Shenker, and I. Stoica, "Tachyon: Reliable, memory speed storage for cluster computing frameworks," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC '14)*. Seattle, WA, USA: ACM, Nov. 2014, pp. 1–15.

[46] H. A. Maruf, H. Wang, A. Dhanotia, J. Weiner, N. Agarwal, P. Bhattacharya, C. Petersen, M. Chowdhury, S. Kanaujia, and P. Chauhan, "TPP: Transparent page placement for CXL-enabled tiered-memory," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '23)*. Vancouver, BC, Canada: ACM, Mar. 2023, pp. 742–755.

[47] W. Meng, B. Lee, and H. Wang, "Understanding bottlenecks for efficiently serving LLM inference with KV offloading," arXiv:2601.19910, 2025.

[48] G. L. Mao and R. Zhou, "Energy characterization of KV cache offloading under agentic workloads," in *HotCarbon Workshop on Sustainable Computer Systems (HotCarbon '26)*, Seattle, WA, USA, Jul. 2026, to appear in ACM SIGENERGY Energy Informatics Review (HotCarbon '26 special issue); volume, issue and DOI not yet assigned as of 2026-10-16. [Online]. Available: https://hotcarbon.org/assets/2026/paper-20.pdf